

/*

nomes matricula participacao

- Victor Alvarenga Hwang - 202208766005 - TA
- Arthur Calebe Carvalho Da Silva - 202508449013 - TA
- Raí Lamper de Avila - 202402627279 - TA
- Matheus Cocenzo e Silva - 202107291052 - TA

Projeto: Sistema Interpretador de Instruções com Sensor de Distância e Atuadores

Placa: Arduino Mega 2560

Versão sem display (saída pelo Serial Monitor)

*/

#include <Keypad.h>

// =====

// ===== ELEMENTOS ARQUITETURAIS OBRIGATÓRIOS =====

// =====

int MEM[16]; // memória de dados simulada

int PC = 0; // program counter

byte IR = 0; // instruction register

int ACC = 0; // acumulador

bool FLAG_Z = false; // flag de comparação

bool EXECUTANDO = true; // controle de execução do programa

// =====

// ===== MEMÓRIA DE PROGRAMA =====

```

// =====

String PROGRAMA[16]; // memória de instruções

int TAM_PROGRAMA = 0; // quantidade de instruções carregadas

int PONTEIRO_CARGA = 0; // próximo endereço livre no LOAD

// =====

// ===== MODOS =====

// =====

bool MODO_LOAD = false;

bool MODO_RUN = false;

// Buffer temporário usado durante a digitação no teclado

String bufferInstrucao = "";

// =====

// ===== PINAGEM =====

// =====

// Sensor HC-SR04

const int PIN_TRIG = 41;

const int PIN_ECHO = 42;

// LEDs

const int LED1 = 2;

```

```
const int LED2 = 3;

const int LED3 = 4;


// Buzzer

const int BUZZER = 45;


// =====

// ===== TECLADO =====

// =====

// O teclado usa os pinos que antes estavam separados para outra função.

// Ajuste apenas se sua montagem física estiver diferente.


const byte LINHAS = 4;

const byte COLUNAS = 4;


char teclas[LINHAS][COLUNAS] = {

    {'1', '2', '3', 'A'},

    {'4', '5', '6', 'B'},

    {'7', '8', '9', 'C'},

    {'*', '0', '#', 'D'}

};


// Linhas e colunas do teclado 4x4

byte pinosLinhas[LINHAS] = {26, 28, 30, 32};

byte pinosColunas[COLUNAS] = {34, 36, 38, 40};
```

```
Keypad teclado = Keypad(makeKeymap(teclas), pinosLinhas, pinosColunas, LINHAS, COLUNAS);
```

```
// =====
```

```
// ===== STRUCT =====
```

```
// =====
```

```
struct InstrucaoDecodificada {
```

```
    int opcode;
```

```
    int operando;
```

```
    bool temOperando;
```

```
    String textoOriginal;
```

```
};
```

```
// =====
```

```
// ===== PROTÓTIPOS =====
```

```
// =====
```

```
void inicializarHardware();
```

```
void inicializarSistema();
```

```
void limparPrograma();
```

```
void limparMemoriaDados();
```

```
void processarTecla(char tecla);
```

```
void entrarModoLoad();
```

```
void sairModoLoad();
```

```
void iniciarRun();
```

```
bool armazenarInstrucao(String texto);
```

```
InstrucaoDecodificada decodificarInstrucao(String texto);
```

```
String nomeMneMonico(int opcode);
```

```
String opcodeBinario4Bits(int opcode);
```

```
bool opcodeExigeOperando(int opcode);
```

```
void executarProximaInstrucao();
```

```
void executarInstrucao(InstrucaoDecodificada inst);
```

```
void mostrarEstadoSerial(String instrucaoExecutada);
```

```
void mostrarProgramaSerial();
```

```
long lerDistanciaCM();
```

```
void ligarLED(int numero);
```

```
void desligarLED(int numero);
```

```
void desligarTodosLEDs();
```

```
void ligarBuzzer();
```

```
void desligarBuzzer();
```

```
void tratarInstrucaoALERT();
```

```
void exibirOpcodeAtualBinario();
```

```

// =====

// ===== SETUP =====

// =====

void setup() {

  Serial.begin(9600);

  inicializarHardware();

  inicializarSistema();


  Serial.println(F("====="));
  Serial.println(F("Sistema Interpretador de Instrucoes - Arduino"));
  Serial.println(F("Versao sem display - saida via Serial Monitor"));
  Serial.println(F("-----"));
  Serial.println(F("Teclas de controle:"));
  Serial.println(F("# -> entra/sai do modo LOAD"));
  Serial.println(F("A -> confirma e armazena a instrucao"));
  Serial.println(F("B -> RUN"));
  Serial.println(F("* -> executa 1 instrucao"));
  Serial.println(F("C -> apaga ultimo caractere"));
  Serial.println(F("D -> separador entre opcode e operando"));
  Serial.println(F("-----"));
  Serial.println(F("Exemplos de digitacao no teclado:"));
  Serial.println(F("LOADK 5 -> 2 D 5 A"));
  Serial.println(F("ADDK 3 -> 3 D 3 A"));
  Serial.println(F("DISP -> 1 0 A"));

```

```
Serial.println(F("HALT   -> 1 5 A"));

Serial.println(F("====="));

}
```

```
// =====

// ===== LOOP =====

// =====
```

```
void loop() {

    char tecla = teclado.getKey();
```

```
    if (tecla) {

        processarTecla(tecla);

    }

}
```

```
// =====

// ===== INICIALIZACAO =====

// =====
```

```
void inicializarHardware() {

    pinMode(PIN_TRIG, OUTPUT);

    pinMode(PIN_ECHO, INPUT);
```

```
    pinMode(LED1, OUTPUT);

    pinMode(LED2, OUTPUT);
```

```
pinMode(LED3, OUTPUT);
```

```
pinMode(BUZZER, OUTPUT);
```

```
desligarTodosLEDs();
```

```
desligarBuzzer();
```

```
}
```

```
void inicializarSistema() {
```

```
    limparMemoriaDados();
```

```
    limparPrograma();
```

```
    PC = 0;
```

```
    IR = 0;
```

```
    ACC = 0;
```

```
    FLAG_Z = false;
```

```
    EXECUTANDO = true;
```

```
    MODO_LOAD = false;
```

```
    MODO_RUN = false;
```

```
    bufferInstrucao = "";
```

```
}
```

```
void limparMemoriaDados() {
```

```
    for (int i = 0; i < 16; i++) {
```

```
        MEM[i] = 0;
```



```
}  
}
```

```
void limparPrograma() {  
    for (int i = 0; i < 16; i++) {  
        PROGRAMA[i] = "";  
    }  
    TAM_PROGRAMA = 0;  
    PONTEIRO_CARGA = 0;  
}
```

```
// =====  
// ===== PROCESSAMENTO DE TECLAS =====  
// =====
```

```
void processarTecla(char tecla) {  
    Serial.print(F("Tecla: "));  
    Serial.println(tecla);  
  
    // # entra ou sai do modo LOAD  
    if (tecla == '#') {  
        if (!MODO_LOAD) {  
            entrarModoLoad();  
        } else {  
            sairModoLoad();  
        }  
    }
```

```
    return;
}

// B = RUN
if (tecla == 'B') {
    iniciarRun();
    return;
}

// * = executa uma instrucao
if (tecla == '*') {
    if (MODO_RUN) {
        executarProximaInstrucao();
    } else {
        Serial.println(F("Nao esta em modo RUN. Pressione B para iniciar."));
    }
    return;
}

// Fora do LOAD, as demais teclas não montam instrução
if (!MODO_LOAD) {
    Serial.println(F("Tecla ignorada. Entre em LOAD com # ou use B/*."));
    return;
}

// A = confirmar / armazenar
```

```
if (tecla == 'A') {  
    if (bufferInstrucao.length() == 0) {  
        Serial.println(F("Nenhuma instrucao digitada."));  
        return;  
    }  
}
```

```
if (armazenarInstrucao(bufferInstrucao)) {  
    bufferInstrucao = "";  
}  
return;  
}
```

// C = apagar ultimo caractere

```
if (tecla == 'C') {  
    if (bufferInstrucao.length() > 0) {  
        bufferInstrucao.remove(bufferInstrucao.length() - 1);  
        Serial.print(F("Buffer atual: "));  
        Serial.println(bufferInstrucao);  
    }  
    return;  
}
```

// D = separador entre opcode e operando

```
if (tecla == 'D') {  
    bufferInstrucao += ' ';  
    Serial.print(F("Buffer atual: "));
```

```
Serial.println(bufferInstrucao);  
return;  
}
```

```
// Digitos
```

```
if (tecla >= '0' && tecla <= '9') {  
    bufferInstrucao += tecla;  
    Serial.print(F("Buffer atual: "));  
    Serial.println(bufferInstrucao);  
    return;  
}
```

```
Serial.println(F("Tecla nao utilizada neste modo."));  
}
```

```
void entrarModoLoad() {  
    MODO_LOAD = true;  
    MODO_RUN = false;  
    EXECUTANDO = true;  
    bufferInstrucao = "";
```

```
limparPrograma();
```

```
PC = 0;
```

```
Serial.println(F("==== MODO LOAD ATIVADO ====="));
```

```
Serial.println(F("Programa limpo."));
```

```
Serial.println(F("Digite instrucoes em decimal."));  
Serial.println(F("Use D como separador e A para confirmar."));  
}
```

```
void sairModoLoad() {  
    MODO_LOAD = false;  
    bufferInstrucao = "";
```

```
    Serial.println(F("==== MODO LOAD ENCERRADO ====="));  
    Serial.print(F("Total de instrucoes armazenadas: "));  
    Serial.println(TAM_PROGRAMA);  
    mostrarProgramaSerial();  
}
```

```
void iniciarRun() {  
    if (TAM_PROGRAMA == 0) {  
        Serial.println(F("Nao ha programa carregado. Use # para entrar em LOAD."));  
        return;  
    }
```

```
    MODO_RUN = true;  
    MODO_LOAD = false;  
    PC = 0;  
    EXECUTANDO = true;
```

```
    Serial.println(F("==== MODO RUN INICIADO ====="));
```

```

    Serial.println(F("Pressione * para executar uma instrucao por vez."));
}

// =====
// ===== ARMAZENAMENTO DE PROGRAMA =====
// =====

bool armazenarInstrucao(String texto) {
    if (PONTEIRO_CARGA >= 16) {
        Serial.println(F("Erro: memoria de programa cheia (max 16 instrucoes)."));
        return false;
    }

    texto.trim();

    InstrucaoDecodificada inst = decodificarInstrucao(texto);

    if (inst.opcode < 0 || inst.opcode > 15) {
        Serial.println(F("Erro: opcode invalido."));
        return false;
    }

    if (opcodeExigeOperando(inst.opcode) && !inst.temOperando) {
        Serial.println(F("Erro: esta instrucao exige operando."));
        return false;
    }
}

```

```
if (!opcodeExigeOperando(inst.opcode) && inst.temOperando) {  
    Serial.println(F("Aviso: operando sera ignorado para esta instrucao."));  
}
```

```
PROGRAMA[PONTEIRO_CARGA] = texto;
```

```
TAM_PROGRAMA++;
```

```
PONTEIRO_CARGA++;
```

```
Serial.print(F("Instrucao armazenada em ["));
```

```
Serial.print(PONTEIRO_CARGA - 1);
```

```
Serial.print(F("]: "));
```

```
Serial.print(nomeMneMonico(inst.opcode));
```

```
if (inst.temOperando && opcodeExigeOperando(inst.opcode)) {
```

```
    Serial.print(F(" "));
```

```
    Serial.print(inst.operando);
```

```
}
```

```
Serial.print(F(" | opcode binario: "));
```

```
Serial.println(opcodeBinario4Bits(inst.opcode));
```

```
return true;
```

```
}
```

```
InstrucaoDecodificada decodificarInstrucao(String texto) {
```

```
InstrucaoDecodificada inst;

inst.opcode = -1;

inst.operando = 0;

inst.temOperando = false;

inst.textoOriginal = texto;


texto.trim();


int posEspaco = texto.indexOf(' ');


if (posEspaco == -1) {
    inst.opcode = texto.toInt();
    inst.temOperando = false;
} else {
    String parteOpcode = texto.substring(0, posEspaco);
    String parteOperando = texto.substring(posEspaco + 1);
    parteOperando.trim();

    inst.opcode = parteOpcode.toInt();
    inst.operando = parteOperando.toInt();
    inst.temOperando = true;
}


return inst;
}
```



```
// =====  
// ===== ISA / OPCODES =====  
// =====
```

```
String nomeMneMonico(int opcode) {
```

```
    switch (opcode) {
```

```
        case 0: return "NOP";
```

```
        case 1: return "READ";
```

```
        case 2: return "LOADK";
```

```
        case 3: return "ADDK";
```

```
        case 4: return "SUBK";
```

```
        case 5: return "CMPK";
```

```
        case 6: return "LEDON";
```

```
        case 7: return "LEDOFF";
```

```
        case 8: return "BUZON";
```

```
        case 9: return "BUZOFF";
```

```
        case 10: return "DISP";
```

```
        case 11: return "ALERT";
```

```
        case 12: return "BINC";
```

```
        case 13: return "STORE";
```

```
        case 14: return "LOADM";
```

```
        case 15: return "HALT";
```

```
        default: return "INVALIDA";
```

```
    }
```

```
}
```

```
String opcodeBinario4Bits(int opcode) {
```

```
    String s = "";
```

```
    for (int i = 3; i >= 0; i--) {
```

```
        s += ((opcode >> i) & 1) ? '1' : '0';
```

```
    }
```

```
    return s;
```

```
}
```

```
bool opcodeExigeOperando(int opcode) {
```

```
    switch (opcode) {
```

```
        case 2:
```

```
        case 3:
```

```
        case 4:
```

```
        case 5:
```

```
        case 6:
```

```
        case 7:
```

```
        case 13:
```

```
        case 14:
```

```
            return true;
```

```
        default:
```

```
            return false;
```

```
    }
```

```
}
```

```
// =====
```

```
// ===== CICLO DE INSTRUCAO / UC =====
```

```
// =====
```

```
void executarProximaInstrucao() {
```

```
    if (!EXECUTANDO) {
```

```
        Serial.println(F("Programa encerrado por HALT. Pressione B para RUN novamente."));
```

```
        return;
```

```
    }
```

```
    if (PC < 0 || PC >= TAM_PROGRAMA) {
```

```
        Serial.println(F("PC fora do limite do programa. Execucao encerrada."));
```

```
        EXECUTANDO = false;
```

```
        MODO_RUN = false;
```

```
        return;
```

```
    }
```

```
// Busca da instrucao na memoria de programa
```

```
String instrucaoTexto = PROGRAMA[PC];
```

```
// Decodificacao
```

```
InstrucaoDecodificada inst = decodificarInstrucao(instrucaoTexto);
```

```
// Carrega opcode no IR
```

```
IR = (byte)inst.opcode;
```

```
// Execucao
```

```
executarInstrucao(inst);
```

```
// Monta nome para exibicao

String nomeInstrucao = nomeMneMonico(inst.opcode);

if (inst.temOperando && opcodeExigeOperando(inst.opcode)) {

    nomeInstrucao += " " + String(inst.operando);

}
```

```
// Mostra estados internos

mostrarEstadoSerial(nomeInstrucao);
```

```
// Atualiza PC, exceto apos HALT

if (EXECUTANDO) {

    PC = PC + 1;

}

}
```

```
void executarInstrucao(InstrucaoDecodificada inst) {

    switch (inst.opcode) {

        case 0: // NOP

            break;

        case 1: // READ

            ACC = (int)lerDistanciaCM();

            break;
```

case 2: // LOADK

ACC = inst.operando;

break;

case 3: // ADDK

ACC = ACC + inst.operando;

break;

case 4: // SUBK

ACC = ACC - inst.operando;

break;

case 5: // CMPK

FLAG_Z = (ACC == inst.operando);

break;

case 6: // LEDON

ligarLED(inst.operando);

break;

case 7: // LEDOFF

desligarLED(inst.operando);

break;

case 8: // BUZON

ligarBuzzer();

```
break;
```

```
case 9: // BUZOFF
```

```
    desligarBuzzer();
```

```
    break;
```

```
case 10: // DISP
```

```
    Serial.print(F("DISP -> ACC = "));
```

```
    Serial.println(ACC);
```

```
    if (ACC > 9) {
```

```
        Serial.println(F("Overflow detectado (valor > 9)."));
```

```
    }
```

```
    if (ACC < 0) {
```

```
        Serial.println(F("Valor negativo detectado."));
```

```
    }
```

```
    break;
```

```
case 11: // ALERT
```

```
    tratarInstrucaoALERT();
```

```
    break;
```

```
case 12: // BINC
```

```
    exhibirOpcodeAtualBinario();
```

```
    break;
```

case 13: // STORE

```
if (inst.operando >= 0 && inst.operando < 16) {  
    MEM[inst.operando] = ACC;  
    Serial.print(F("STORE -> MEM["));  
    Serial.print(inst.operando);  
    Serial.print(F("] = "));  
    Serial.println(ACC);  
} else {  
    Serial.println(F("Erro: endereco de memoria invalido em STORE."));  
}  
break;
```

case 14: // LOADM

```
if (inst.operando >= 0 && inst.operando < 16) {  
    ACC = MEM[inst.operando];  
    Serial.print(F("LOADM -> ACC recebeu MEM["));  
    Serial.print(inst.operando);  
    Serial.print(F("] = "));  
    Serial.println(ACC);  
} else {  
    Serial.println(F("Erro: endereco de memoria invalido em LOADM."));  
}  
break;
```

case 15: // HALT

```
EXECUTANDO = false;

MODO_RUN = false;

Serial.println(F("HALT processado. Execucao encerrada."));

break;
```

```
default:
```

```
Serial.println(F("Erro: opcode invalido."));

EXECUTANDO = false;

MODO_RUN = false;

break;
```

```
}
```

```
}
```

```
// =====
```

```
// ===== SERIAL / ESTADO =====
```

```
// =====
```

```
void mostrarEstadoSerial(String instrucaoExecutada) {
```

```
Serial.println(F("-----"));
```

```
Serial.print(F("PC: "));
```

```
Serial.print(PC);
```

```
Serial.print(F(" | IR: "));
```

```
Serial.print(instrucaoExecutada);
```

```
Serial.print(F(" | ACC: "));
```

```
Serial.print(ACC);
```

```
Serial.print(F(" | FLAG_Z: "));
```



```
Serial.println(FLAG_Z ? 1 : 0);
```

```
Serial.print(F("IR opcode binario: "));
```

```
Serial.println(opcodeBinario4Bits(IR));
```

```
Serial.print(F("MEM: "));
```

```
for (int i = 0; i < 16; i++) {
```

```
    Serial.print(F("[");
```

```
    Serial.print(i);
```

```
    Serial.print(F("]="));
```

```
    Serial.print(MEM[i]);
```

```
    if (i < 15) Serial.print(F(" "));
```

```
}
```

```
Serial.println();
```

```
Serial.println(F("-----"));
```

```
}
```

```
void mostrarProgramaSerial() {
```

```
    Serial.println(F("Programa armazenado:"));
```

```
    for (int i = 0; i < TAM_PROGRAMA; i++) {
```

```
        InstrucaoDecodificada inst = decodificarInstrucao(PROGRAMA[i]);
```

```
        Serial.print(F("[");
```

```
        Serial.print(i);
```

```
        Serial.print(F("] "));
```

```
        Serial.print(nomeMneMonico(inst.opcode));
```

```
        if (inst.temOperando && opcodeExigeOperando(inst.opcode)) {
```

```

    Serial.print(F(" "));
    Serial.print(inst.operando);
}
Serial.print(F(" | opcode: "));
Serial.println(opcodeBinario4Bits(inst.opcode));
}
}

// =====
// ===== SENSOR =====
// =====

long lerDistanciaCM() {
    long duracao;
    float distancia;

    digitalWrite(PIN_TRIG, LOW);
    delayMicroseconds(5);

    digitalWrite(PIN_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);

    duracao = pulseIn(PIN_ECHO, HIGH, 30000);

    if (duracao == 0) {

```

```
Serial.println(F("READ: sem retorno do sensor."));  
return -1;  
}
```

```
distancia = duracao * 0.0343 / 2.0;
```

```
Serial.print(F("READ -> distancia lida: "));
```

```
Serial.print(distancia);
```

```
Serial.println(F(" cm"));
```

```
return (long)distancia;
```

```
}
```

```
// =====
```

```
// ===== LEDS =====
```

```
// =====
```

```
void ligarLED(int numero) {
```

```
    switch (numero) {
```

```
        case 1:
```

```
            digitalWrite(LED1, HIGH);
```

```
            Serial.println(F("LED 1 ligado."));
```

```
            break;
```

```
        case 2:
```

```
            digitalWrite(LED2, HIGH);
```

```
            Serial.println(F("LED 2 ligado."));
```

```
        break;
    case 3:
        digitalWrite(LED3, HIGH);
        Serial.println(F("LED 3 ligado.));
        break;
    default:
        Serial.println(F("Erro: LED invalido. Use 1, 2 ou 3.));
        break;
}
}
```

```
void desligarLED(int numero) {
    switch (numero) {
        case 1:
            digitalWrite(LED1, LOW);
            Serial.println(F("LED 1 desligado.));
            break;
        case 2:
            digitalWrite(LED2, LOW);
            Serial.println(F("LED 2 desligado.));
            break;
        case 3:
            digitalWrite(LED3, LOW);
            Serial.println(F("LED 3 desligado.));
            break;
        default:
```

```

        Serial.println(F("Erro: LED invalido. Use 1, 2 ou 3."));
        break;
    }
}

void desligarTodosLEDs() {
    digitalWrite(LED1, LOW);
    digitalWrite(LED2, LOW);
    digitalWrite(LED3, LOW);
}

// =====

// ===== BUZZER =====

// =====

void ligarBuzzer() {
    digitalWrite(BUZZER, HIGH);
    Serial.println(F("Buzzer ligado."));
}

void desligarBuzzer() {
    digitalWrite(BUZZER, LOW);
    Serial.println(F("Buzzer desligado."));
}

// =====

```

```

// ===== ALERT =====

// =====

void tratarInstrucaoALERT() {
    long distancia = lerDistanciaCM();

    if (distancia < 0) {
        Serial.println(F("ALERT: leitura invalida do sensor."));
        desligarTodosLEDs();
        desligarBuzzer();
        return;
    }

    if (distancia < 10) {
        Serial.println(F("ALERT: distancia < 10 cm -> LED de alerta + buzzer"));
        ligarLED(1);
        desligarLED(2);
        desligarLED(3);
        ligarBuzzer();
    }

    else if (distancia < 20) {
        Serial.println(F("ALERT: 10 cm <= distancia < 20 cm -> LED de alerta, sem buzzer"));
        ligarLED(1);
        desligarLED(2);
        desligarLED(3);
        desligarBuzzer();
    }
}

```

```
}  
else {  
    Serial.println(F("ALERT: distancia >= 20 cm -> alertas desligados"));  
    desligarLED(1);  
    desligarLED(2);  
    desligarLED(3);  
    desligarBuzzer();  
}  
}  
  
// =====  
// ===== BINC =====  
// =====  
  
void exibirOpcodeAtualBinario() {  
    Serial.print(F("Opcode binario da instrucao atual (IR): "));  
    Serial.println(opcodeBinario4Bits(IR));  
}
```